

ESTRUTURA DE DADOS

Pilha (Stack / LIFO)

Material de Estudo com Simulado

Contexto / Motivação

Uma pilha é uma estrutura de dados linear onde inserção e remoção ocorrem apenas pelo mesmo extremo — o topo. O princípio LIFO (Last In, First Out — o último a entrar é o primeiro a sair) modela naturalmente: o histórico de desfazer (Ctrl+Z), a pilha de chamadas de funções do processador, a avaliação de expressões aritméticas, e o backtracking em algoritmos de busca.

```
push(30) → TOP
  | 30 | <- topo
  | 20 |
  | 10 |
  ■■■■■

pop() retorna 30:
  | 20 | <- novo topo
  | 10 |
  ■■■■■
```

Figura 1 — Pilha com 3 elementos. *push()* adiciona no topo, *pop()* remove do topo.

Dica: Pilha de chamadas (call stack): ao chamar `func()`, o SO empilha o endereço de retorno e as variáveis locais. Ao retornar, desempilha (`pop`). Recursão infinita causa Stack Overflow — a pilha excede seu limite de memória.

Implementação em C99

Estrutura e criação

```
// AA08 - Pilha (LIFO) com lista encadeada
// Autor: Gabriel Gonçalves
// FATEC Ourinhos – Estrutura de Dados

#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int dado;
    struct No *proximo;
} No;

typedef struct {
    No *topo; // único ponteiro necessário
    int tamanho;
} Pilha;

Pilha* pilha_criar(void) {
    Pilha *p = malloc(sizeof(Pilha));
    p->topo = NULL;
    p->tamanho = 0;
    return p;
}

int pilha_vazia(const Pilha *p) {
    return p->topo == NULL;
}
```

Código 1 — Pilha com lista encadeada. Só o topo importa: push e pop são $O(1)$.

Push, Pop e Peek

```
// Push: insere no topo — O(1)
void pilha_push(Pilha *p, int dado) {
    No *novo      = malloc(sizeof(No));
    novo->dado     = dado;
    novo->proximo  = p->topo; // encadeia com topo atual
    p->topo        = novo;    // novo vira topo
    p->tamanho++;
}

// Pop: remove e retorna o topo — O(1)
int pilha_pop(Pilha *p) {
    if (pilha_vazia(p)) {
        fprintf(stderr, "Erro: pop em pilha vazia (underflow)\n");
        exit(EXIT_FAILURE);
    }
    No *temp = p->topo;
    int dado = temp->dado;
    p->topo   = temp->proximo; // avança o topo
    free(temp);               // libera o nó removido
    p->tamanho--;
    return dado;
}

// Peek: consulta o topo SEM remover — O(1)
int pilha_peek(const Pilha *p) {
    if (pilha_vazia(p)) {
        fprintf(stderr, "Erro: peek em pilha vazia\n");
        exit(EXIT_FAILURE);
    }
    return p->topo->dado;
}
```

Código 2 — push(), pop() e peek() são todas O(1). Notar tratamento de underflow.

Destruição e Uso Completo

```
void pilha_destruir(Pilha *p) {
    while (!pilha_vazia(p))
        pilha_pop(p);    // reutiliza pop para liberar cada nó
    free(p);
}

int main(void) {
    Pilha *p = pilha_criar();

    pilha_push(p, 10);
    pilha_push(p, 20);
    pilha_push(p, 30);

    printf("Topo: %d\n", pilha_peek(p));    // 30
    printf("Pop:  %d\n", pilha_pop(p));    // 30
    printf("Pop:  %d\n", pilha_pop(p));    // 20
    printf("Tam:  %d\n", p->tamanho);      // 1

    pilha_destruir(p);
    return 0;
}
```

Código 3 — Programa completo. pilha_destruir() usa pop() internamente: elegante e correto.

Complexidade e Comparativo

Operação	Complexidade	Observação
push()	O(1)	Insere no topo — sem percorrer
pop()	O(1)	Remove do topo — sem percorrer
peek()	O(1)	Lê p->topo->dado diretamente
vazia()	O(1)	Verifica p->topo == NULL
destruir()	O(n)	Precisa remover todos os nós

Critério	Pilha (array)	Pilha (lista enc.)
Tamanho	Fixo (definido na criação)	Dinâmico (ilimitado)
Overhead	Nenhum (acesso por índice)	8 bytes por nó (ponteiro)
Cache-friendly	Sim (dados contíguos)	Nao (fragmentado na heap)
Uso indicado	Tamanho conhecido e pequeno	Tamanho desconhecido

SIMULADO

Questão 1 (Código)

Complete a função pilha_push abaixo. Os campos da struct Pilha são: No *topo e int tamanho. A struct No tem: int dado e No *proximo. void pilha_push(Pilha *p, int dado) { No *novo = malloc(sizeof(No)); novo->dado = dado; novo->proximo = _____; p->topo = _____; _____; }

// Escreva sua resposta aqui:

Questão 2 (Código)

A função pilha_pop abaixo tem um bug de vazamento de memória (memory leak). Identifique e corrija o bug. int pilha_pop(Pilha *p) { if (pilha_vazia(p)) return -1; int dado = p->topo->dado; p->topo = p->topo->proximo; p->tamanho--; return dado; }

// Escreva sua resposta aqui:

Questão 3 (Conceitual)

Explique o princípio LIFO com um exemplo real do dia a dia. Por que a pilha de chamadas do processador usa exatamente esse princípio?

Questão 4 (Raciocínio)

Execute as operações abaixo em ordem e escreva o estado da pilha (do topo para a base) após cada operação: push(5), push(3), push(8), pop(), push(1), peek() Qual é o valor retornado por peek()?

Questão 5 (Conceitual)

Qual das estruturas abaixo seria a mais indicada para implementar a funcionalidade de 'desfazer' (Ctrl+Z) em um editor de texto? Justifique. a) Array de tamanho fixo b) Pilha com lista encadeada c) Fila (FIFO) d) Lista duplamente encadeada

GABARITO COMENTADO

Questão 1 (Código):

novo->proximo = p->topo; (encadeia com topo atual) / p->topo = novo; (novo vira o topo) / p->tamanho++; (incrementa contador). Ordem importa: encadear ANTES de atualizar topo.

Questão 2 (Código):

Bug: o nó removido não é liberado. Antes de avançar o topo, guardar referência: No *temp = p->topo; depois p->topo = temp->proximo; e free(temp); No código original, o nó antigo do topo perde referência e vaza memória.

Questão 3 (Conceitual):

LIFO = Last In, First Out. Exemplo: pilha de pratos — o último prato colocado é o primeiro a ser retirado. Call stack: ao chamar f() que chama g(), g() é empilhada por último e termina primeiro, liberando contexto para f() continuar. LIFO garante que a função mais interna (mais recente) seja a primeira a retornar — ordem correta de desempilhamento do contexto.

Questão 4 (Raciocínio):

push(5): [5] / push(3): [3,5] / push(8): [8,3,5] / pop() retorna 8: [3,5] / push(1): [1,3,5] / peek() retorna 1 (topo sem remover): estado final [1,3,5]. peek() retorna 1.

Questão 5 (Conceitual):

b) Pilha com lista encadeada. Ctrl+Z desfaz a ação MAIS RECENTE — exatamente LIFO. Lista encadeada permite crescimento ilimitado (número de desfazimentos não é conhecido). Fila (FIFO) desfaria a ação mais antiga, o que seria incorreto. Array fixo limitaria o número de desfazimentos.